

Person A Write-up — Bike Rental Demand Forecasting

Name: Dhruv Talavat (PES1UG24AM087) **Scope:** Dataset understanding, EDA, preprocessing, feature engineering, baseline models — the first half of the ML pipeline. **Reference paper:** Du, He, Zhechev — *Forecasting Bike Rental Demand* (Kaggle bike-sharing competition, RMSLE metric). Our featurization and metric choices are aligned with it so Person B's advanced models can be compared on the same basis.

1. Problem Statement

Urban bike-sharing systems must balance fleet supply with hour-to-hour demand. Too few bikes at rush hour means lost rides and frustrated commuters; too many bikes parked at low-demand hours wastes dock space and rebalancing effort. As the reference paper notes, limited bike supply, rising demand, and the cost of storing and relocating bikes are exactly the factors that motivate demand forecasting.

Goal: predict the number of bike rentals (`cnt`) for a given hour, using calendar information (hour, weekday, month, season, holiday/working day) and weather conditions (temperature, feels-like temperature, humidity, windspeed). This is a **supervised regression** task on hourly data from the Capital Bikeshare system (Washington D.C., 2011–2012).

Primary metric — RMSLE. Following the reference paper (which competed on Kaggle with the same dataset), our headline metric is the **Root Mean Squared Logarithmic Error**:

$$\text{RMSLE} = \sqrt{\text{mean}(\log(p_i + 1) - \log(a_i + 1))^2)}$$

RMSLE penalizes relative errors rather than absolute ones, so under-predicting the 3 AM demand (small counts) is not punished as harshly as under-predicting the 6 PM rush. We also report RMSE, MAE and R^2 for completeness (the team-level success criteria agreed with Person B).

2. Dataset

Property	Value
Source	UCI ML Repository — Bike Sharing Dataset (hourly file, <code>hour.csv</code>)
Rows	17,379 hourly records
Period	2011-01-01 to 2012-12-31
Target	<code>cnt</code> = <code>casual</code> + <code>registered</code> (rentals per hour)
Features	Calendar (<code>season</code> , <code>yr</code> , <code>mnth</code> , <code>hr</code> , <code>weekday</code> , <code>holiday</code> , <code>workingday</code>) + weather (<code>weathersit</code> , <code>temp</code> , <code>atemp</code> , <code>hum</code> , <code>windspeed</code>)

Note: the reference paper used the *Kaggle* variant of the dataset (10,886 train rows — first 20 days of each month — and 6,493 test rows), which is a different split of the same underlying Capital Bikeshare data. We use the full UCI `hour.csv` and create our own temporal split, which is the time-series-correct approach the paper itself lists as future work ("a realistic model should only use data observed prior to the time of prediction").

Column dictionary (Person A's subset)

Column	Type	Meaning
dteday	date	Calendar day
season	int 1-4	1=Winter, 2=Spring, 3=Summer, 4=Fall
yr	int 0-1	0=2011, 1=2012
mnth	int 1-12	Month
hr	int 0-23	Hour of day
holiday	binary	Public holiday flag
weekday	int 0-6	Day of week
workingday	binary	1 if not weekend/holiday
weathersit	int 1-4	1=Clear, 2=Cloudy/Mist, 3=Light rain/snow, 4=Heavy rain/snow
temp	float	Normalized air temperature ($\div 41$ °C)
atemp	float	Normalized feels-like temperature ($\div 50$ °C)
hum	float	Normalized humidity ($\div 100$)
windspeed	float	Normalized wind speed ($\div 67$)
casual	int	Non-registered user count — excluded from features (leakage)
registered	int	Registered user count — excluded from features (leakage)
cnt	int	Target: total rentals

Data quality checks performed

- **Missing values:** none — all 17,379 rows are complete.
- **Duplicates:** no duplicated rows on (dteday, hr).
- **Target range:** 1 to 977 rentals/hour, mean ≈ 189 , right-skewed (skew ≈ 1.28) — expected for count data; this skew is also why RMSLE (log-scale) is a sensible headline metric, as the paper observed.

3. EDA — What the data says

Figures live in reports/, produced by `src/data_understanding.py`.

Temporal patterns

- `02_daily_trend.png` — rentals grow year-over-year from 2011 to 2012, with dips during winter and extreme-weather days.
- `03_hourly_box.png` — the **bimodal daily pattern**: commuter rush-hour peaks around 8 AM and 5–6 PM on working days, flatter midday-heavy demand on weekends.
- `12_workingday_hourly_profile.png` — *directly validates the paper's peak-hour observation*: weekday peaks fall between 7–9 AM and 5–7 PM; weekend demand is broad from 10 AM to 6 PM. This figure is the evidence behind our `peak_time` feature.
- `11_monthly_by_year.png` — both years peak in summer (Jun–Sep), trough in Dec–Feb.

Weather effects

- 06_atemp_scatter.png — positive relationship with feels-like temperature, plateauing in the hot region.
- 07_hum_scatter.png — negative relationship; muggy hours suppress riding.
- 08_windspeed_scatter.png — weak effect, mostly negative at high wind speeds.
- 09_correlation_heatmap.png — quantifies the above against cnt, and shows temp↔atemp collinearity (≈ 0.99) that motivates dropping raw temp.

Weather situation

- 05_weather_avg.png — demand drops sharply from Clear → Cloudy → Wet. Severe is almost never observed, so predictions in that regime are extrapolation.

User composition (context, not a feature)

- 10_casual_vs_registered.png — registered users commute on weekdays; casual users ride weekends and nice weather. The paper's §5.5 tried regressing on the two user types separately and found it *worse* than modeling total cnt — consistent with our decision to keep them out of features entirely (they also leak the target).

4. Preprocessing

Implemented in `src/preprocess_and_baseline.py`, aligned with the paper's §2.2 featurization experiments:

1. **Date decomposition** — like the paper, we decompose the timestamp into `mnth` (kept as a feature — the paper replaced season with month and found it useful), weekday (the paper's "day of week" feature), and `hr`.
2. **Numeric scaling** — `StandardScaler` inside a `ColumnTransformer` for the linear baseline; Random Forest gets passthrough (trees don't need scaling).
3. **Categorical encoding** — `OneHotEncoder(handle_unknown='ignore')`, matching the paper's practice of converting categoricals to binary indicators for linear models. Unknown categories at inference map to all-zeros.
4. **Collinearity handling — drop temp, keep atemp** — exactly the paper's experiment: temp and atemp are ≈ 0.99 correlated; we keep atemp because "it feels like" is what matters to a rider's decision.
5. **Leakage policy** — `casual`, `registered`, `instant`, raw `dteday` are never features. `cnt = casual + registered` by definition, so any of the three leaks the target.

The paper's remaining featurization ideas — discretizing continuous temperature into buckets, dropping/keeping `holiday`, adding peak-hour indicators — are covered in the Feature Engineering section below.

5. Feature Engineering (paper-aligned)

Feature	Purpose	Paper analogue
<code>temp_bucket</code>	Discretize feels-like temperature into 4 quantile buckets (cold/mild/warm/hot) to help linear models handle the non-linear temp response	"Discretizing continuous variables" (§2.2, bullet 1)

Feature	Purpose	Paper analogue
peak_time	3-way bucket: weekday_peak (7–9 AM, 5–7 PM), weekend_peak (10 AM–6 PM), or off-peak — validated by our EDA figure 12	"Adding peak hour indicator variables" (§2.2, bullet 6)
hour_sin, hour_cos	Cyclic hour encoding so 23:00 is numerically near 00:00	modern complement to the paper's hour feature
dayofweek_sin/cos	Cyclic weekday encoding	the paper's "day of week" feature (§2.2, bullet 3)
month_sin, month_cos	Cyclic month encoding	the paper's "replace season with month" idea (§2.2, bullet 2)
is_rush_hour	Hard flag for weekday commuting peaks	same motivation as peak_time, simpler form
is_weekend	Quick Weekend/weekday distinction	derived from workingday

Holiday variable — kept, deliberately. The paper dropped holiday after seeing no visible effect. In the full UCI dataset holidays are rare (~1.9% of rows — far fewer than in the Kaggle split), and our tests showed it carries signal for weekend-like demand on midweek holidays. We keep it, matching the paper's principle of *testing* each featurization idea rather than assuming it.

6. Train/Test Split (Temporal)

Unlike the Kaggle split (arbitrary 20-days-of-month split), we use the split the paper recommends as future work: **by calendar date**. Train on the earliest ~80% of days (2011-01-01 → 2012-08-07), test on the final ~20% (2012-08-08 → 2012-12-31) — 13,915 train / 3,464 test rows. The model never sees data from after its test window.

7. Baseline Models (Person A deliverable)

The paper tested 7 models (Linear, GLMNet, GBM, PCR, SVR, RF, CTree) — the *advanced* ones (GBM, SVR, stacking etc.) are Person B's half. Our baselines give Person B three things to beat:

1. **Linear Regression (OLS)** on scaled numerics + one-hot categoricals — the paper's §3.1 baseline.
2. **Random Forest Regressor** — the paper's best non-final model (10-fold CV RMSLE 0.503). We run a small grid over `n_estimators` (200/400), `max_depth` (12/18/None), `min_samples_leaf` (1/3/5) with `TimeSeriesSplit(4)`.

Results on the held-out temporal test set

Model	RMSLE (primary)	RMSE	MAE	R ²
Linear Regression	1.050	119.47	90.54	0.706
Random Forest (best)	0.410	79.70	52.67	0.869

Best model per the script: **Random Forest** (`max_depth=None`, `min_samples_leaf=1`, `n_estimators=400`).

Comparison with the reference paper: our RF's test RMSLE (0.410) is in the same range as the paper's best CV models (CTree 0.460, RF 0.503). Direct comparison is not exact — different (tougher, time-

ordered) test split, no leaderboard, and our feature set — but the pattern matches: **tree-based models dominate, plain linear models lag**. The paper's variable-importance finding also reproduces in our data: hour-related features (`hour_sin/cos`, `peak_time`, `is_rush_hour`) are by far the most predictive, weather features contribute a smaller but real effect.

Artifacts produced for Person B

- `models/baseline_model.pkl` — fitted RF pipeline (fallback model for the demo; Person B's advanced models should beat it on RMSLE).
- `models/baseline_metrics.json` — chosen baseline's test metrics (RMSLE + RMSE + MAE + R^2).
- `models/baseline_diagnostics.json` — model name, hyperparameters, full feature list, CV score, split date, paper-alignment notes.
- `reports/13_baseline_pred_vs_actual.png`, `reports/14_baseline_residuals.png` — visual comparison.

8. Conclusions (Person A half)

- The dataset is clean (no missing/duplicated rows) but demand is strongly seasonal, hourly and weather-driven: hour of day is by far the strongest signal, with weather contributing a smaller, consistent effect — the same conclusion the reference paper reached from its variable-importance study.
- Paper-aligned feature engineering — peak-hour buckets (weekday 7–9 AM & 5–7 PM, weekend 10 AM–6 PM), quantile temperature buckets, cyclic hour/weekday/month encodings, and dropping collinear raw temp — plus a leakage-safe temporal split produced a strong baseline.
- **Random Forest is the Person A baseline to beat: RMSLE 0.410, RMSE 79.7, R^2 0.869** on the held-out 2012-08-08 → 2012-12-31 period, versus Linear Regression's RMSLE 1.050 / R^2 0.706. This reproduces the reference paper's central finding that tree-based models substantially outperform plain linear models on this data.
- Deliverables from this half: reproducible EDA pipeline (12 figures), the preprocessing/feature-engineering module, baseline artifacts for comparison, a working Streamlit input interface, and this write-up plus the slide deck.
- **Open for Person B:** advanced models (GBM/SVR/CTree/stacking per paper §3) must beat RMSLE 0.410 on the same temporal split; conclusions are completed after that comparison in Person B's results section.

9. Limitations / Handoff Notes

- **Split difference:** the paper's Kaggle split is not time-ordered; ours is. Expect Person B's CV-vs-test gap to be smaller per §5.4 of the paper but absolute errors to differ from leaderboard numbers.
- `weathersit == 4` (severe weather) has almost no rows — predictions in that regime are extrapolation, not signal.
- Two years only; year-over-year growth is folded into the `yr` feature rather than a growth model. The paper's §7.3 time-series suggestion (train only on the recent past) is a good Person B extension.
- Random Forest can't extrapolate beyond its training range; per the paper's §3.4, gradient boosting (GBM) may handle the recent-trend drift better.
- The paper's §5.5 finding stands: don't regress `casual` and `registered` separately — both our correlation audit and the paper found total-count modeling better and cleaner.
- User-type columns (`casual`, `registered`) are excluded everywhere (leakage); documented in §2.